

MIZAN

ميزان

Product Requirements Document

Internal Business Management & Inventory System — Version 1

Document version	1.0 — Design Baseline
Status	Approved for implementation
Date	13 September 2026
Product owner	Ayob Milad
Technology	Laravel 13 · Blade · Tailwind 4 · Alpine.js · MySQL 8
Supersedes	Requirements & Business Rules V1 (10 Sep) Use Cases PRD V1 (12 Sep) ERD & Database Design Decisions

Contents

1. Purpose and Document Authority

2. System Overview and Scope

3. Actors and Responsibilities

4. Global Business Rules

5. Resolved Design Conflicts

C1 Sales session and warehouse · C2 Technology stack · C3 Purchase tables

C4 Payment methods · C5 Opening cash float · C6 Return warehouse

C7 Return structure · C8 Role model

6. Supplementary Design Decisions

7. Functional Requirements

Identity · Catalog · Inventory · Purchasing · Selling · Returns · Reporting

8. Data Model — Entity Relationship Design

Identity and governance · Master data · Inventory core

Selling · Purchasing · Inventory operations · Returns

9. Technical Architecture

10. Security Requirements

11. Out of Scope — V2 Roadmap

12. Appendix A — Permission Catalogue

13. Appendix B — Role / Permission Matrix

14. Appendix C — Decision Log

1. Purpose and Document Authority

This document is the single authoritative specification for **Mizan V1**. It consolidates three earlier documents — the Requirements & Business Rules paper, the Use Cases PRD, and the ERD Design Decisions paper — and resolves the contradictions that existed between them.

Authority. Where this document differs from any earlier paper, **this document governs**. Section 5 lists every point on which the earlier documents disagreed, the decision taken, and the rules that were consequently cancelled. Rules cancelled here must not be reintroduced during implementation.

1.1 Design philosophy

Mizan is not a CRUD application. Its difficulty lies in six properties that must hold under concurrent, real-world use:

Property	What it demands of the implementation
Transactional integrity	Every business operation that touches more than one row succeeds completely or not at all.
Inventory traceability	No quantity ever changes without a stock movement recording the cause.
Concurrency safety	Correctness is guaranteed by database locks, never by application-level checks alone.
Auditability	Sensitive actions answer: who, what, when, where, which record, what changed, why.
Accountability	Every operation carries the identity of the employee who performed it.
Historical integrity	Completed transactions are immutable. Corrections are new operations, never edits.

The system must be able to answer, for any unit of stock: who sold it, from which warehouse, during which session, which invoice caused the decrease, why inventory increased, who performed an adjustment, why money was refunded, and how much cash an employee should hold at shift close.

2. System Overview and Scope

Mizan is an internal business management and inventory system for a retail business operating multiple warehouses across different cities. V1 serves **one company**; it is not multi-tenant.

2.1 In scope

Domain	Capabilities
Identity	Authentication, invitation-based employee onboarding, roles, permissions
Master data	Products, product units, categories, brands, warehouses, customers, suppliers
Inventory	Per-warehouse stock balances, stock movements, adjustments, inter-warehouse transfers
Purchasing	Purchase orders, order confirmation, goods receiving
Selling	Sales sessions, sales, invoices, multi-warehouse selling, walk-in customers
Returns	Multi-item returns against a sale, cash refunds, session reconciliation impact
Oversight	Reports, audit log, system settings

2.2 Out of scope for V1

Delivery representatives; customer accounts and customer portal; order and delivery tracking; real-time delivery status; customer-facing communication or chat; WebSocket infrastructure; online payment gateway integration; product variants; multi-tenancy; partial goods receiving; discounts at line-item level.

Extensibility constraint. V1 must remain structurally open to these features. Specific provisions are made in the data model: purchase receipts are separated from purchase orders so partial receiving becomes a constraint removal rather than a migration; the refund table carries a `method` column although only `CASH` is valid in V1.

3. Actors and Responsibilities

Actor	Can do	Cannot do
Admin	Everything. Manages employees, roles, permissions, system settings, and reviews the audit log.	—
Manager	Manages products, categories, brands, suppliers, customers and warehouses. Creates and confirms purchase orders. Views inventory, stock movements, all sales and all reports.	Manage users, roles, permissions or settings. Create sales. Adjust or transfer stock. Receive goods.
Sales Employee	Opens and closes own sales session, creates sales from any warehouse, creates and edits customers, views own sales, executes returns and cash refunds.	View other employees' sales. Modify or delete completed sales. Deactivate or delete customers. Touch inventory operations.
Warehouse Employee	Views inventory and stock movements, receives purchases, adjusts stock, transfers stock between warehouses.	Create sales. Manage users, roles or permissions. Delete inventory records.
Customer	A business entity only — exists to be associated with sales.	Has no login, password, role, permissions or dashboard in V1.

Separation of duties. The Manager reviews but does not execute. A manager who could also adjust stock could conceal a shortage and then approve his own correction. Where a business genuinely needs one person to do both, the correct answer is a new named role holding both permission sets — an explicit, auditable decision — not a weakening of the default roles.

4. Global Business Rules

ID	Rule
BR-GLOBAL-01	Users must be authenticated before accessing any protected operation.
BR-GLOBAL-02	Every protected operation is authorised on the backend against the user's role permissions. Frontend permission checks exist for user experience only and never replace backend authorisation.
BR-GLOBAL-03	The backend is the source of truth. It calculates and validates prices, totals, taxes, discounts, stock quantities, warehouse access, the active sales session, refund amounts and permissions. No value in this list may be trusted from the client.
BR-GLOBAL-04	Records with historical transactions are never hard-deleted. Deactivation via a <code>status</code> column is used instead. Soft deletes are explicitly rejected (see §6.9).
BR-GLOBAL-05	Any operation that changes multiple related records executes inside a single database transaction: sale creation, purchase receiving, stock adjustment, stock transfer, return and refund.
BR-GLOBAL-06	Sensitive and inventory-related operations are recorded in the audit log with actor, action, entity, previous value, new value, reason where applicable, and timestamp.
BR-GLOBAL-07	Stock quantity may never become negative, by any operation, including manual adjustment. This is enforced both in application logic and by database constraint.

BR-GLOBAL-08

Inventory rows participating in a critical read-check-write sequence must be locked with `SELECT . . . FOR UPDATE` for the duration of the transaction.

5. Resolved Design Conflicts

The source documents contradicted each other on eight points. Each is resolved below. The **CANCELLED** rules are void and must not appear in the implementation.

C1 — Sales session and warehouse

Conflict. The Requirements paper bound a session to exactly one warehouse (BR-SESSION-05/06/07/08). The Use Cases paper bound the session to the user only, with each sale carrying its own warehouse (UC-06, UC-07).

Decision. The session belongs to the **user only**. It has no warehouse. Every sale stores its own `warehouse_id`. An employee may sell from different warehouses within one session; changing the selected warehouse never affects sales already recorded.

Consequences.

- `sales_sessions` has no `warehouse_id` column.
- `sales.warehouse_id` is mandatory and validated on *every* sale request, not once per session.
- **CANCELLED** BR-SESSION-05, BR-SESSION-06, BR-SESSION-07, BR-SESSION-08.

C2 — Technology stack

Conflict. Both source papers specified a GraphQL API with a Vue front end.

Decision. V1 is a single Laravel application using **Blade + Tailwind CSS + Alpine.js**. No GraphQL, no Vue. The service layer, business rules, transaction boundaries and locking strategy are unchanged — only the presentation layer differs. A future API layer would sit on top of the same services.

C3 — Purchase tables

Conflict. The Use Cases paper defined purchase orders with `DRAFT` / `CONFIRMED` / `RECEIVED` statuses, but the ERD table list contained only `purchase_receipts` — no `purchases` or `purchase_items`.

Decision. Four tables. The order and the receipt are separate entities.

<code>purchases</code>	→ the order placed with the supplier
<code>purchase_items</code>	→ what was ordered, at what cost
↓	
<code>purchase_receipts</code>	→ the physical receiving event
<code>purchase_receipt_items</code>	→ what actually arrived
↓	
<code>stock_movements</code>	→ reference = <code>purchase_receipt</code> (never the order)

V1 permits exactly one receipt per purchase, enforced by a unique constraint. V2 partial receiving is enabled by *removing that one constraint* — no data migration.

C4 — Payment methods and return eligibility

Conflict. The reconciliation example distinguished *Gross Sales* from *Cash Sales*, implying non-cash payment, yet no payment method appeared anywhere in the sale structure.

Decision.

- `sales.payment_method` is an enum: `CASH`, `CARD`, `TRANSFER`.
- **NEW RULE** A return is permitted only when the original sale's payment method is `CASH`. Sales paid by card or transfer cannot be returned in V1. The backend rejects such a request; the UI hides the return action.
- Refund method remains `CASH` (BR-REFUND-04 stands).

Why this rule matters. Refunding a card sale in cash removes money from a drawer that never received it, producing a shift shortage the employee did not cause. Restricting returns to cash sales eliminates that class of reconciliation error entirely.

C5 — Opening cash float

Gap. Reconciliation was specified as `Cash Sales - Cash Refunds`, with no starting balance. A refund processed today against yesterday's cash sale would drive expected cash negative.

Decision. **NEW** `sales_sessions.opening_float`, counted and entered by the employee at session start.

Expected Cash = Opening Float + Cash Sales - Cash Refunds
Difference = Actual Cash - Expected Cash

A difference does not prevent closure but is always recorded.

C6 — Destination warehouse for returned stock

Gap. UC-12 said inventory increases in "the relevant warehouse", which C1 left undefined.

Decision. Returned stock always re-enters the **original sale's warehouse**. Inventory is therefore perfectly symmetric: what left a warehouse returns to it, and the `SALE / RETURN` movement pair reconciles per warehouse.

Where goods are physically returned to a different branch, the correct handling is a return to the original warehouse followed by an explicit **stock transfer** — two traceable operations rather than one invisible one. The return screen must display which warehouse receives the stock.

C7 — Return structure

Conflict. The flows described returning a single sale item; the entity model showed `Return → Return Items` with one refund.

Decision. A return is a **multi-item** document: `returns + return_items`, mirroring `sales / sale_items`. Exactly **one refund per return** (1:1), created in the same transaction and enforced by a unique constraint on `refunds.return_id`.

C8 — Role model

Conflict. Both papers stated "a User belongs to a Role" (singular), yet the ERD listed a `role_user` pivot table implying many-to-many.

Decision. **One role per user** via `users.role_id`. **CANCELLED** `role_user`.

Rationale: many-to-many roles make a user's effective permissions an unnamed union that nobody inspects as a whole, and it renders the `ROLE_CHANGED` audit entry meaningless. A user who performs two jobs is given a single role named for that combined job.

6. Supplementary Design Decisions

6.1 Multi-unit products NEW

Goods are purchased wholesale (cartons) and sold retail (pieces). A product therefore has several units of measure with conversion factors and independent prices.

Governing rule — no exceptions. Stock is always stored in the **base unit**. Every operation converts to the base unit before touching inventory.

Product: Pepsi 330ml

Unit	factor	is_base	purchase price	selling price
piece	1	yes	0.800	1.500
dozen	12	no	9.000	16.000
carton	24	no	17.000	30.000

Prices are stored *per unit row*, never derived by multiplication: a carton at 17.000 is cheaper per piece than 24 individual pieces at 0.800, which is how wholesale pricing actually works.

Every transaction line stores `product_unit_id`, a **frozen** `unit_factor`, the `quantity` in the chosen unit, and the computed `base_quantity`. Freezing the factor is mandatory for the same reason cost is frozen: if a carton is later redefined from 24 to 36, historical documents must continue to mean what they meant when issued.

Profit calculation across units

unit_price	30.000	price of one carton
quantity	2	cartons sold
line_total	60.000	
base_quantity	48	pieces deducted from stock
unit_cost	0.708	cost per BASE unit

Profit = line_total - (base_quantity × unit_cost)
= 60.000 - (48 × 0.708) = 26.000

6.2 Discounts NEW

- Discount is applied at **invoice level**, entered by the seller as a percentage, default 0.
- Both `discount_percent` and the computed `discount_amount` are stored, so a printed invoice and a report can never disagree through rounding.
- The discount is distributed across lines and the resulting `net_unit_price` is stored per line.
- Refunds are calculated on the net (post-discount) price.** A piece sold at 1000 with a 20% invoice discount refunds 800 — what the customer actually paid.

6.3 Tax

Sales carry `subtotal`, `tax_amount` and `total`. Tax is NOT NULL DEFAULT 0 — not nullable. A null money column propagates through `total = subtotal + tax_amount` and silently corrupts `SUM()` in reports; zero is the mathematically correct representation of "no tax".

6.4 Money and quantity precision

Concern	Decision
Money columns	<code>decimal(15,3)</code> — the Libyan Dinar has three decimal places (dirham)

Quantity columns	<code>decimal(12,3)</code> — supports unit conversion factors and future weight/length products
Floating point	Never used for any monetary or quantity value
Historical prices	Frozen on <code>sale_items</code> and <code>purchase_items</code> at transaction time

6.5 Warehouse access

There is **no** user-to-warehouse assignment table. Following C1, a sales employee serves from whichever warehouse holds stock, so restricting them would contradict the session model. Warehouses carry `supervisor_id`, which is **informational**, not a constraint: making it a constraint would halt a warehouse whenever its supervisor is absent, and accountability is already captured by the acting user recorded on every operation.

6.6 Document numbering

Human-facing references (`INV`, `PO`, `ADJ`, `TR`, `RET`) are issued from a single `document_sequences` table, read under a row lock inside the operation's transaction. Deriving a number from `MAX(...)` + 1 is rejected: two concurrent sales would receive the same invoice number.

```
lock row (type, year) → increment → INV-2026-00543 → commit
```

Invoice number is a column on `sales`; there is no separate invoice entity.

6.7 Categories

Flat. No parent-child nesting in V1.

6.8 Customers

`sales.customer_id` is nullable — walk-in sales without a recorded customer are permitted. Where a customer is recorded, `phone` is unique when provided (MySQL permits unlimited NULLs under a unique index, which is exactly the required semantic).

6.9 Deletion strategy

Laravel `SoftDeletes` is rejected. A `status` enum (`active` / `inactive`) is used instead. Soft deletes silently exclude rows from every query, which is dangerous in an inventory system: a "deleted" product may still have physical stock on a shelf.

6.10 Language and presentation

The interface is **Arabic, right-to-left**. Product, customer and warehouse names are entered in Arabic. SKUs, document references and all numerals remain Latin and are rendered with directional isolation and tabular figures so they neither reverse inside Arabic text nor break column alignment in financial tables.

7. Functional Requirements

7.1 Employee onboarding (UC-14)

Admin creates account → system issues invitation → employee opens link
→ employee sets own password → account becomes active

- The admin never sets an employee password.
- The invitation token is temporary, expiring and single-use.
- A user cannot authenticate while `status = pending` or `disabled`.
- Employees are disabled, never deleted; historical transactions retain their link to the employee.

7.2 Roles and permissions (UC-15)

- Permissions are system-defined constants, seeded, never created or deleted through the interface.
- Roles are configurable and hold any set of permissions.
- Changing a role's permissions immediately affects every user holding that role.
- A role cannot be deleted while any user is assigned to it, nor if it is a system role.
- Role changes are audited as `ROLE_CHANGED` with old and new values.

7.3 Products (UC-10)

- SKU is required and unique. Each sellable SKU is an independent product; there are no product variants in V1.
- Creating a product never creates stock. Stock enters only through receiving, adjustment, transfer or return.
- Every product has exactly one base unit; additional units carry a conversion factor and their own prices.
- Inactive products cannot be sold.
- Price changes never alter historical `sale_items` or `purchase_items` prices.
- `reorder_level` (in base units) drives the stock status indicator: `0` → `out`, `≤ reorder_level` → `low`, otherwise `available`.

7.4 Create sale (UC-01)

```
authenticated user
→ backend resolves the user's OPEN session      (never trusts a client session id)
→ validate warehouse, products active, quantities > 0
→ BEGIN TRANSACTION
    → lock warehouse_stocks rows                  (SELECT ... FOR UPDATE)
    → verify availability in base units
    → calculate prices, discount distribution, tax, total
    → issue invoice number from document_sequences
    → insert sale + sale_items (freezing unit_price, unit_cost, unit_factor, net_unit_price)
    → decrement warehouse_stocks
    → insert stock_movements (SALE, negative, reference = sale)
→ COMMIT
→ return invoice
```

Rejections: no open session; insufficient stock; inactive product; unauthorised warehouse. On any failure the entire transaction rolls back — no partial sale, no stock change, no movement rows.

7.5 Sales session (UC-06, UC-08)

- A sales employee may hold **at most one open session**, enforced at database level (§8.15).
- Opening requires the employee to count and enter the opening float.
- Every sale and every return is attached to the session open at the time it was created.

- Session totals are always derived from recorded transactions, never entered manually.
- Closing records expected cash, actual cash, the difference and optional notes. A difference never blocks closure.
- A closed session is never reopened and never receives new sales. Post-closure corrections are a separate, authorised, audited workflow that amends reconciliation data only — never the underlying sales.

7.6 Purchasing (UC-02, UC-03)

- A purchase order belongs to one supplier and holds one or more items with ordered quantity and unit cost.
- Statuses: `draft` → `confirmed` → `received`, plus `cancelled` **ADDED** so abandoned orders do not remain open indefinitely.
- Creating or confirming a purchase does **not** change inventory.
- Only a confirmed, not-yet-received purchase can be received. Receiving selects the destination warehouse.
- Receiving increments stock and writes `PURCHASE_RECEIPT` movements inside one transaction, then sets the order to `received`. A purchase cannot be received twice.

7.7 Stock adjustment (UC-04)

- Used when counted physical quantity differs from system quantity. A reason is mandatory.
- `difference = actual - current`, computed by the backend.
- Recorded per line: previous quantity, new quantity, difference. Plus header: warehouse, user, reason, timestamp.
- Adjustment, stock update, movement and audit entry are one transaction. Resulting stock may never be negative.
- Completed adjustments are never edited or deleted.

7.8 Stock transfer (UC-05)

- Source and destination warehouses must differ — enforced by database constraint.
- Source is locked and verified; destination row is locked or safely created.
- Produces exactly two movements per line: `TRANSFER_OUT` (negative) and `TRANSFER_IN` (positive).
- Partial transfers are impossible: the whole transfer commits or rolls back.

7.9 Returns and refunds (UC-12, UC-13)

- Every return references exactly one completed sale whose payment method is `CASH`.
- Returnable quantity per line = `sale_items.quantity - sale_items.returned_quantity`. The sale item row is locked while this is verified and incremented, so concurrent returns cannot exceed what was sold.
- A reason is mandatory on every return.
- The return is attached to the employee's open session and affects that session's reconciliation.
- Refund amount is computed server-side from net line prices; the client cannot propose an amount.
- Return, stock increase, movement, refund and audit entry are a single transaction.
- The original sale is never modified. Completed returns and refunds are never edited or deleted.

7.10 Stock movements (UC-17)

Every inventory change, without exception, creates a movement. Movements are historical and read-only.

Type	Sign	Reference
<code>PURCHASE_RECEIPT</code>	positive	<code>purchase_receipt</code>
<code>SALE</code>	negative	<code>sale</code>
<code>RETURN</code>	positive	<code>return</code>
<code>ADJUSTMENT</code>	either	<code>stock_adjustment</code>

TRANSFER_OUT	negative	stock_transfer
TRANSFER_IN	positive	stock_transfer

7.11 Reporting (UC-16)

Sales, purchases, inventory, profit, stock movement, sales-employee performance and session reports. All read-only, all sourced from authoritative business records, all filtered by the viewer's permissions. Profit reporting relies on `sale_items.unit_cost` captured at sale time — without it, changing a product's purchase price would silently rewrite last year's profit.

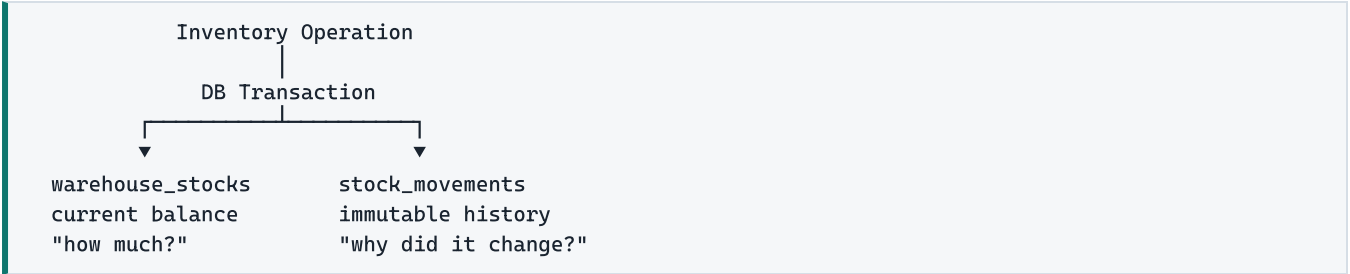
7.12 System settings (UC-18)

Company identity, currency, tax rate, invoice prefix and notification preferences. Settings marked critical are validated and audited on change. Authentication and authorisation logic is never exposed as a business setting, and no setting change may alter the meaning of historical invoices.

8. Data Model — Entity Relationship Design

Twenty-six tables. All money is `decimal(15,3)`; all quantities are `decimal(12,3)`. All foreign keys are `RESTRICT` on delete unless stated, in support of BR-GLOBAL-04; only true child rows cascade.

8.1 Architectural separation



Current stock is **stored**, not recomputed by summing movements. Movements provide the audit trail; `balance_after` on each movement lets the two be reconciled at any time.

Identity and governance

1 roles

Column	Type	Notes
id	bigint unsigned PK	
name	varchar(50) UNIQUE	Display name
slug	varchar(50) UNIQUE	admin, manager, sales_employee, warehouse_employee
description	varchar(255) NULL	
is_system	boolean DEFAULT 0	System roles cannot be deleted
timestamps		

2 permissions

Column	Type	Notes
id	bigint unsigned PK	
name	varchar(100) UNIQUE	e.g. <code>sales.create</code>
group	varchar(50)	Grouping for the permission UI
description	varchar(255) NULL	

No timestamps — seeded system constants.

3 permission_role

Column	Type	Notes
role_id	FK roles CASCADE	Composite primary key, no id, no timestamps
permission_id	FK permissions CASCADE	

4 users

Column	Type	Notes
id	bigint unsigned PK	
role_id	FK roles RESTRICT	Exactly one role per user (C8)
name	varchar(100)	
email	varchar(150) UNIQUE	
phone	varchar(20) NULL	
password	varchar(255) NULL	NULL until the invitation is accepted
status	enum pending / active / disabled	Login rejected unless active
last_login_at	timestamp NULL	
remember_token	varchar(100) NULL	
timestamps		Index: role_id, status

5 user_invitations

Column	Type	Notes
id	bigint unsigned PK	
user_id	FK users CASCADE	
token_hash	varchar(64) UNIQUE	SHA-256 of the token, never the token itself
expires_at	timestamp	Expired token is rejected
accepted_at	timestamp NULL	Non-null enforces single use
invited_by	FK users RESTRICT	
timestamps		

6 audit_logs

Column	Type	Notes
id	bigint unsigned PK	
user_id	FK users RESTRICT NULL	NULL for system-originated events
action	varchar(50)	ROLE_CHANGED, STOCK_ADJUSTED, REFUND_CREATED ...
auditable_type	varchar(100)	Polymorphic target. Index (type, id)
auditable_id	bigint unsigned	
old_values	json NULL	
new_values	json NULL	
reason	varchar(255) NULL	
ip_address	varchar(45) NULL	
terminal_name	varchar(50) NULL	Which point of sale performed the action
created_at	timestamp	No updated_at — immutable by nature

7 settings

Column	Type	Notes
id	bigint unsigned PK	
key	varchar(100) UNIQUE	company_name, currency, tax_rate, invoice_prefix ...
value	text NULL	
type	enum string / integer / decimal / boolean / json	Cast target
group	varchar(50)	
is_critical	boolean DEFAULT 0	Critical changes are audited
updated_by	FK users RESTRICT NULL	
timestamps		

Master data

8 warehouses

Column	Type	Notes
id	bigint unsigned PK	
name	varchar(100)	
code	varchar(20) UNIQUE	TRP, BNG ...
city / address / phone	varchar NULL	
supervisor_id	FK users RESTRICT NULL	Informational, not an access constraint (\$6.5)
status	enum active / inactive	
timestamps		

9 categories · 10 brands

Column	Type	Notes
id	bigint unsigned PK	
name	varchar(100) UNIQUE	
description	varchar(255) NULL	categories only
status	enum active / inactive	Flat — no parent_id (\$6.7)
timestamps		

11 products

Column	Type	Notes
id	bigint unsigned PK	
sku	varchar(50) UNIQUE	Required; Latin characters
name	varchar(150)	Arabic
category_id	FK categories RESTRICT NULL	
brand_id	FK brands RESTRICT NULL	
description	text NULL	
reorder_level	decimal(12,3) DEFAULT 0	In base units; drives low-stock status
status	enum active / inactive	Inactive products cannot be sold
timestamps		Index: status, category_id, brand_id, name

Prices live on `product_units`, not here.

12 product_units NEW

Column	Type	Notes
id	bigint unsigned PK	
product_id	FK products CASCADE	

name	varchar(30)	piece / dozen / carton. UNIQUE (product_id, name)
factor	decimal(12,4)	How many base units this unit contains
is_base	boolean DEFAULT 0	Base unit has factor = 1
barcode	varchar(50) UNIQUE NULL	Each unit may carry its own barcode
purchase_price	decimal(15,3) DEFAULT 0	Explicit per unit — never derived by multiplication
selling_price	decimal(15,3) DEFAULT 0	
status	enum active / inactive	
base_key	generated STORED, UNIQUE	IF(is_base, product_id, NULL) — guarantees exactly one base unit per product

13 customers

Column	Type	Notes
id	bigint unsigned PK	
name	varchar(100)	Required
phone	varchar(20) UNIQUE NULL	Unique when provided
email / address	varchar NULL	
notes	text NULL	
status	enum active / inactive	
created_by	FK users RESTRICT NULL	Which employee registered this customer
timestamps		

14 suppliers

Column	Type	Notes
id	bigint unsigned PK	
name	varchar(100)	Required
contact_person	varchar(100) NULL	
phone / email / address	varchar NULL	
notes	text NULL	
status	enum active / inactive	Inactive suppliers cannot receive new purchases
timestamps		

Inventory core

15 warehouse_stocks

Column	Type	Notes
id	bigint unsigned PK	
warehouse_id	FK warehouses RESTRICT	UNIQUE (warehouse_id, product_id)
product_id	FK products RESTRICT	
quantity	decimal(12,3) unsigned DEFAULT 0	Always in base units. CHECK (quantity >= 0)
timestamps		Index: product_id

Defence in depth. unsigned plus a CHECK constraint makes negative stock impossible at the storage layer. The application check remains, to produce a comprehensible error message — but the database guarantees the invariant even if a code path is wrong.

16 stock_movements

Column	Type	Notes
id	bigint unsigned PK	
product_id	FK products RESTRICT	
warehouse_id	FK warehouses RESTRICT	
movement_type	enum (6 types)	See §7.10
quantity	decimal(12,3) <i>signed</i>	Positive = in, negative = out. Base units only
balance_after	decimal(12,3) unsigned	Balance immediately after this movement
reference_type	varchar(100)	Polymorphic source. Index (type, id)
reference_id	bigint unsigned	
user_id	FK users RESTRICT	
reason	varchar(255) NULL	
created_at	timestamp	No updated_at. Index (product, warehouse, created_at)

Polymorphic trade-off. The database cannot enforce referential integrity on `reference_id`, because its target table varies. This is accepted deliberately — the alternative is five nullable foreign keys. The mitigation is mandatory: **stock movements are created exclusively through a single service**. No other code path may insert into this table.

17 document_sequences NEW

Column	Type	Notes
document_type	varchar(20)	Composite primary key
year	smallint unsigned	
last_number	int unsigned DEFAULT 0	Incremented under row lock inside the caller's transaction

Selling

18 sales_sessions

Column	Type	Notes
id	bigint unsigned PK	
user_id	FK users RESTRICT	No warehouse_id (C1)
status	enum open / closed	
opening_float	decimal(15,3) DEFAULT 0	Counted by the employee at start (C5)
opened_at	timestamp	
closed_at	timestamp NULL	
expected_cash	decimal(15,3) NULL	Derived at close from transactions
actual_cash	decimal(15,3) NULL	Entered by the employee
difference	decimal(15,3) NULL	Backend-calculated; recorded, never hidden
closing_notes	text NULL	
active_key	generated STORED, UNIQUE	IF(status='open', user_id, NULL)

One open session, guaranteed. A generated column holds the user id only while the session is open, under a unique index. Because MySQL permits unlimited NULLs in a unique index, closed sessions are unconstrained while a second open session is physically impossible — even if two requests arrive simultaneously and both pass the application check.

19 sales

Column	Type	Notes
id	bigint unsigned PK	
invoice_number	varchar(30) UNIQUE	INV-2026-00001, from document_sequences
sales_session_id	FK sales_sessions RESTRICT	Resolved by the backend, never from the client
user_id	FK users RESTRICT	The selling employee
customer_id	FK customers RESTRICT NULL	NULL = walk-in sale
warehouse_id	FK warehouses RESTRICT	Mandatory per sale (C1)
payment_method	enum CASH / CARD / TRANSFER	Returns permitted only when CASH (C4)
subtotal	decimal(15,3)	
discount_percent	decimal(5,2) DEFAULT 0	Both stored — percentage and computed amount
discount_amount	decimal(15,3) DEFAULT 0	
tax_amount	decimal(15,3) NOT NULL DEFAULT 0	Never nullable (§6.3)
total	decimal(15,3)	
notes	varchar(255) NULL	

timestamps		Index: session, user, customer, warehouse, created_at, payment_method
------------	--	---

20 sale_items

Column	Type	Notes
id	bigint unsigned PK	
sale_id	FK sales CASCADE	
product_id	FK products RESTRICT	
product_unit_id	FK product_units RESTRICT	The unit actually sold
unit_factor	decimal(12,4)	Frozen at sale time (\$6.1)
quantity	decimal(12,3)	In the selected unit
base_quantity	decimal(12,3)	quantity × unit_factor — what stock is deducted by
unit_price	decimal(15,3)	Price of one selected unit, frozen
unit_cost	decimal(15,3)	Cost per base unit, frozen — profit reporting
discount_share	decimal(15,3) DEFAULT 0	This line's share of the invoice discount
net_unit_price	decimal(15,3)	Post-discount price — the basis for refunds
line_total	decimal(15,3)	
returned_quantity	decimal(12,3) DEFAULT 0	Running counter; locked during return validation

Purchasing

21 purchases · 22 purchase_items

Column	Type	Notes
purchases		
reference_number	varchar(30) UNIQUE	PO-2026-00001
supplier_id	FK suppliers RESTRICT	
status	enum draft / confirmed / received / cancelled	cancelled added in this revision
subtotal, total	decimal(15,3)	
created_by	FK users RESTRICT	
confirmed_by / confirmed_at	FK users NULL / timestamp NULL	
purchase_items		
purchase_id	FK purchases CASCADE	
product_id	FK products RESTRICT	
product_unit_id, unit_factor	FK / decimal(12,4)	Purchasing unit, frozen
quantity, base_quantity	decimal(12,3)	
unit_cost, line_total	decimal(15,3)	Historical values

23 purchase_receipts · 24 purchase_receipt_items

Column	Type	Notes
purchase_receipts		
purchase_id	FK purchases RESTRICT	UNIQUE — one receipt per order in V1
warehouse_id	FK warehouses RESTRICT	Destination chosen at receiving
received_by	FK users RESTRICT	
received_at	timestamp	
purchase_receipt_items		
purchase_receipt_id	FK CASCADE	
product_id, product_unit_id	FK RESTRICT	
unit_factor, quantity, base_quantity	decimal	base_quantity increments stock

V2 migration path: drop the unique constraint on `purchase_receipts.purchase_id` and partial receiving works. No schema change, no data migration.

Inventory operations

25 stock_adjustments · stock_adjustment_items

Column	Type	Notes
stock_adjustments		
reference_number	varchar(30) UNIQUE	ADJ-2026-00001
warehouse_id	FK warehouses RESTRICT	
user_id	FK users RESTRICT	
reason	varchar(255) NOT NULL	Mandatory
created_at	timestamp	Immutable
stock_adjustment_items		
product_id, product_unit_id, unit_factor	FK / decimal	
previous_quantity	decimal(12,3)	All in base units; difference is signed
new_quantity	decimal(12,3)	
difference	decimal(12,3) signed	

26 stock_transfers · stock_transfer_items

Column	Type	Notes
stock_transfers		
reference_number	varchar(30) UNIQUE	TR-2026-00001
from_warehouse_id	FK warehouses RESTRICT	CHECK (from_warehouse_id <> to_warehouse_id)
to_warehouse_id	FK warehouses RESTRICT	
user_id	FK users RESTRICT	
created_at	timestamp	Immutable
stock_transfer_items		
product_id, product_unit_id, unit_factor	FK / decimal	
quantity, base_quantity	decimal(12,3)	Produces TRANSFER_OUT and TRANSFER_IN

Returns

27 returns · return_items · refunds

Column	Type	Notes
returns		
reference_number	varchar(30) UNIQUE	RET-2026-00001
sale_id	FK sales RESTRICT	Must be a CASH sale (C4)
sales_session_id	FK sales_sessions RESTRICT	Affects that session's reconciliation

user_id	FK users RESTRICT	
reason	varchar(255) NOT NULL	Mandatory
total_amount	decimal(15,3)	Server-calculated
created_at	timestamp	No warehouse_id — derived from the sale (C6, 3NF)
return_items		
return_id	FK returns CASCADE	
sale_item_id	FK sale_items RESTRICT	Row locked while validating returnable quantity
product_id, product_unit_id, unit_factor	FK / decimal	
quantity, base_quantity	decimal(12,3)	
unit_price	decimal(15,3)	= the sale item's <code>net_unit_price</code>
line_total	decimal(15,3)	
refunds		
return_id	FK returns RESTRICT UNIQUE	Enforces the 1:1 rule (C7)
amount	decimal(15,3)	Server-calculated; client cannot propose a value
method	enum CASH DEFAULT CASH	Column exists for V2 extensibility
user_id, sales_session_id	FK RESTRICT	
created_at	timestamp	Immutable; every refund is audited

9. Technical Architecture

9.1 Stack

Layer	Choice
Framework	Laravel 13, PHP 8.4
Presentation	Blade templates, Tailwind CSS 4, Alpine.js 3 — Arabic RTL
Typography	Cairo, self-hosted (no external CDN dependency at request time)
Database	MySQL 8.0 — required for enforced CHECK constraints and indexed generated columns
Business logic	Service classes owning transaction boundaries; controllers stay thin
Database user	Dedicated least-privilege account scoped to the application schema; never <code>root</code>

9.2 Deployment

Warehouses are located in different cities, which rules out a LAN server and a per-branch installation. V1 is deployed as a **single web application on an internet-accessible server**, with all branches connecting by browser.

Rejected: per-machine installation. A standalone install per terminal implies a database per terminal, which destroys every guarantee in this document — two cashiers would each sell the same last unit. Multi-user operation requires one shared database, without exception.

Rejected for V1: branch-local servers with synchronisation. Reconciling independent databases (conflicting edits, duplicated invoice numbers, divergent stock) is a larger engineering problem than this system itself.

Operational requirements: automated off-site daily backup; a clear, non-silent failure state when connectivity is lost — the application must never allow a sale it cannot record.

9.3 Concurrency

```
DB::transaction(function () use ($warehouseId, $productId, $qty) {
    $stock = WarehouseStock::query()
        ->where('warehouse_id', $warehouseId)
        ->where('product_id', $productId)
        ->lockForUpdate()           // row lock held until commit
        ->firstOrFail();

    if ($stock->quantity < $qty) {
        throw new InsufficientStockException();
    }

    $stock->decrement('quantity', $qty);
    // + stock movement, in the same transaction
});
```

Rows are locked in a consistent order across all operations to avoid deadlocks. The same discipline applies to `sale_items.returned_quantity`, to `document_sequences`, and to both sides of a transfer.

9.4 Idempotency

Operations that a user may submit twice — create sale, receive purchase, execute refund — are protected by an idempotency key supplied with the request and enforced by a unique constraint, so a double submission produces one business record rather than two.

9.5 Data integrity summary

Invariant	Enforcement
Stock never negative	<code>unsigned</code> + <code>CHECK</code> + application validation
One open session per employee	Generated column + unique index
Exactly one base unit per product	Generated column + unique index
One receipt per purchase (V1)	Unique constraint
One refund per return	Unique constraint
Transfer source $\neq$ destination	<code>CHECK</code> constraint
One stock row per product per warehouse	Composite unique constraint
Unique invoice numbers under load	Locked sequence row inside the transaction
No orphaned historical references	<code>RESTRICT</code> foreign keys + status-based deactivation

10. Security Requirements

Area	Requirement
Password storage	Hashed with bcrypt or argon2 — deliberately slow, salted. Never stored or logged in plain text.
Invitation tokens	Cryptographically random, transmitted once in the invitation link, stored only as a SHA-256 hash. A database leak must not permit account activation. Compared in constant time.
Authorisation	Enforced server-side on every protected operation. Client-side permission state is presentational only.
Session resolution	The active sales session is always derived from the authenticated user; a session id supplied by the client is never trusted.
Monetary calculation	Totals, discounts, taxes and refund amounts are computed server-side from stored data only.
Database access	The application connects as a least-privilege user limited to its own schema.
Audit coverage	Role changes, account status changes, stock adjustments, transfers, returns, refunds and critical setting changes.
Secrets	Credentials live in the environment file, excluded from version control. No credential is ever committed.

11. Out of Scope — V2 Roadmap

Feature	Notes on V1 readiness
Partial goods receiving	Remove one unique constraint; structure already supports it
Non-cash refunds	<code>refunds.method</code> already exists as an enum
Product variants	Would introduce a variant layer above products; deliberately avoided in V1
Delivery representatives, customer accounts and portal	Requires customer authentication and an API layer over the existing services
Order and delivery tracking, real-time status	Requires WebSocket infrastructure
Online payment gateways	Requires payment records beyond the single method field

Multi-tenancy	Would require tenant scoping across every query, index and constraint
Line-level discounts	Line already carries <code>discount_share</code> and <code>net_unit_price</code>

Appendix A — Permission Catalogue

Permissions are seeded system constants. They are never created, edited or deleted through the interface.

USERS users.view users.create users.update users.disable	INVENTORY inventory.view inventory.adjust inventory.transfer stock_movements.view
ROLES roles.view roles.create roles.update roles.delete	PURCHASING purchases.view purchases.create purchases.confirm purchases.receive
CATALOG products.view products.create products.update products.deactivate categories.manage brands.manage	SELLING sessions.start sessions.close sessions.view_all sales.view_own sales.create sales.view_all returns.create returns.view_own returns.view_all
PARTNERS suppliers.view suppliers.create suppliers.update suppliers.deactivate customers.view customers.create customers.update customers.deactivate	REPORTS reports.sales reports.purchases reports.inventory reports.profit reports.movements
WAREHOUSES warehouses.view warehouses.create warehouses.update warehouses.deactivate	GOVERNANCE settings.view settings.update audit.view

Appendix B — Role / Permission Matrix

Permission group	Admin	Manager	Sales Employee	Warehouse Employee
Users, roles	Full	—	—	—
Settings, audit log	Full	—	—	—
Products, categories, brands	Full	Full	View	View
Suppliers	Full	Full	—	View
Customers	Full	Full	View, create, update	—
Warehouses	Full	Full	—	View
Inventory view, movements	Full	View	View	View
Adjust, transfer stock	Full	—	—	Full
Purchase orders	Full	Create, confirm	—	View
Receive goods	Full	—	—	Full
Sales sessions	View all	View all	Own: start, close	—
Create sales	Full	—	Full	—
View sales	All	All	Own only	—
Returns and refunds	Full	View all	Create, view own	—
Reports	All	All	—	—

Bold dashes mark the separation-of-duties boundaries described in §3.

Appendix C — Decision Log

#	Question	Decision
C1	Session bound to a warehouse?	No. Session belongs to the user; each sale carries its warehouse. BR-SESSION-05/06/07/08 cancelled
C2	GraphQL + Vue, or Blade?	Blade + Tailwind + Alpine.js. No GraphQL, no Vue in V1
C3	Purchase table structure	Four tables: order and receipt separated
C4	Payment methods	CASH / CARD / TRANSFER; returns allowed only on CASH sales
C5	Opening cash float	Added; entered by the employee at session start
C6	Warehouse receiving returned stock	The original sale's warehouse, always
C7	Return granularity	Multi-item return; exactly one refund per return
C8	Roles per user	One. <code>users.role_id</code> ; <code>role_user</code> cancelled
D1	Missing tables	Added categories, brands, audit_logs, settings, user_invitations, document_sequences, product_units
D2	User-to-warehouse assignment	Rejected. <code>warehouses.supervisor_id</code> is informational only
D3	Negative stock	Never permitted, by any operation
D4	Invoice identity	Column on <code>sales</code> , issued from a locked sequence
D5	Money precision	<code>decimal(15,3)</code> — Libyan Dinar
D6	Profit basis	<code>sale_items.unit_cost</code> frozen at sale time
D7	Tax	Present, NOT NULL DEFAULT 0
D8	Discounts	Invoice-level percentage; refunds use post-discount price
D9	Interface language	Arabic RTL; SKUs and numerals Latin
D10	Deletion	<code>status</code> column; SoftDeletes rejected
D11	Units of measure	Multi-unit with frozen conversion factors and per-unit prices
D12	Quantity type	<code>decimal(12,3)</code>
D13	Category hierarchy	Flat
D14	Walk-in sales	Permitted; <code>customer_id</code> nullable
D15	Purchase cancellation	<code>cancelled</code> status added
D16	Deployment	Single internet-hosted web application; branches span cities
D17	Movement balance	<code>balance_after</code> stored on every movement